

Version 2.0

January 2026



OPERATING SYSTEMS PRACTICUM

MODULE 5 - SHELL PROGRAMMING AND SYSTEM AUTOMATION (BASH SCRIPTING)

Author:

Ir. Denar Regata Akbi, S.Kom., M.Kom.

Putri Nayla Sabri

Muhammad Zamzam Baihaqi

INFORMATICS LABORATORY

University of Muhammadiyah Malang



TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE TARGETS.....	3
PREPARATION.....	3
KEYWORDS.....	3
THEORETICAL BACKGROUND.....	4
1. Introduction to Shell Scripting.....	4
1.1 What is Shell Scripting.....	4
1.2 Why Shell Scripting?.....	4
2. Bash Script Basics.....	4
2.1 Script Structure.....	4
EXERCISE 1 : The Script Foundation (Structure & Variables).....	6
2.2 Variables and Data Handling.....	6
2.3 Input and Output.....	8
3. Control Structures in Bash.....	10
3.1 Conditional Statements.....	10
EXERCISE 2: The Logic Master (Conditionals).....	11
3.2 Looping Structures.....	11
EXERCISE 3: The Loop Specialist (Automation & Lists).....	12
3.3 Basic Functions.....	12
4. Practical Shell Scripting and Automation.....	12
4.1 Script Arguments.....	12
4.2 Common Automation Tasks.....	13
EXERCISE 4: The System Monitor (Arguments & Functions).....	14
4.3 Debugging and Best Practices.....	14
4.4 Auto-run Scripts (System Automation).....	15
EXERCISE 5: The Automator (Cron & Automation).....	16
PRACTICAL APPLICATIONS.....	17
PRACTICE.....	17
DEMO - CASE STUDY.....	17
Demo Rules.....	17
ASSESSMENT RUBRIC.....	19
ASSESSMENT SCALE.....	19



INTRODUCTION

OBJECTIVES

1. Understand basic Bash shell scripting concepts
2. Create and run simple Bash scripts
3. Use variables, input/output, and script arguments
4. Apply basic automation and debugging techniques

MODULE TARGETS

1. Write executable Bash scripts
2. Implement conditionals and loops
3. Automate simple system tasks
4. Debug scripts using basic Bash tools

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا

2. Laptop or personal computer
3. Rocky Linux
4. Virtual Box
5. Strong motivation and commitment to learning

KEYWORDS

Bash, Shell Scripting, Variables, Input/Output, Script Arguments, Automation, Debugging





THEORETICAL BACKGROUND

1. Introduction to Shell Scripting

1.1 What is Shell Scripting

Shell scripting is the process of writing a series of commands in a text file that can be executed by the shell as a program. Instead of typing commands one by one in the terminal, a shell script allows these commands to be grouped and executed automatically in sequence. This makes shell scripting an efficient way to perform tasks repeatedly and consistently.

In most Linux distributions, **Bash (Bourne Again Shell)** is the default shell. Bash acts as an interface between the user and the operating system, interpreting commands entered by the user or read from a script file. Because Bash is widely available and backward compatible with earlier shells, it has become the standard choice for shell scripting in Linux environments.

There is an important distinction between an **interactive shell** and a **shell script**. In an interactive shell, commands are executed immediately as the user types them into the terminal. In contrast, a shell script contains predefined commands stored in a file, which are executed as a unit when the script is run. This difference allows scripts to automate tasks without requiring constant user interaction.

1.2 Why Shell Scripting?

Shell scripting is mainly used for the **automation of repetitive tasks**. Tasks such as creating files, checking disk usage, managing permissions, or backing up directories often follow the same pattern. Writing a script allows these tasks to be completed automatically, saving time and reducing mistakes caused by manual input.

In **system administration**, shell scripts are widely used to manage users, configure file permissions, monitor system resources, and process log files. Scripts help administrators apply consistent operations across systems and quickly respond to routine maintenance needs.

Shell scripting is closely connected to regular shell commands. A shell script is essentially a collection of commands that could be executed manually in the terminal. The difference is that scripts organize these commands into a reusable and automated form. This makes shell scripting a natural extension of basic command-line skills.

2. Bash Script Basics

2.1 Script Structure

A Bash script is a text file that contains a sequence of commands executed by the shell. The foundation of every script is defined by how it is identified and authorized by the system.





A. The Shebang (#!)

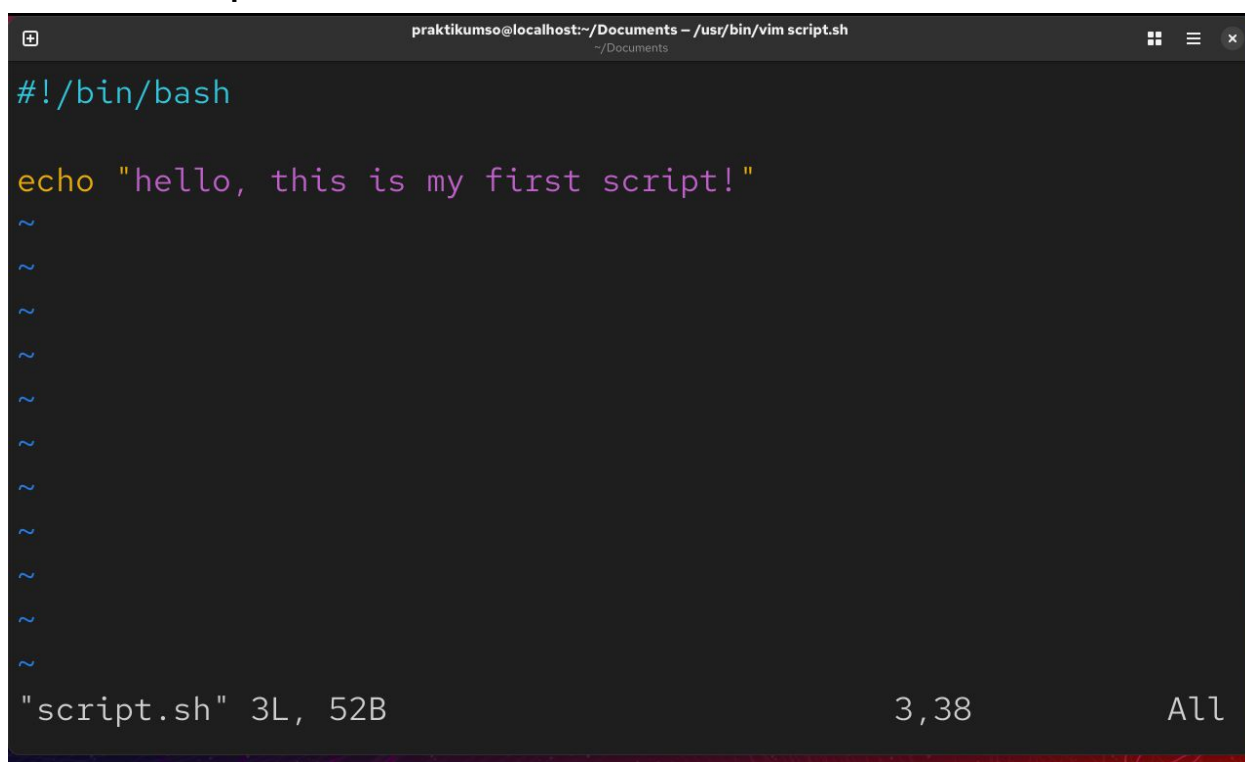
The first and most important part of a script is the shebang, which tells the system which interpreter should be used to run the script. This line must be placed at the very top of the script.

The standard shebang for Linux is:

```
#!/bin/bash
```

This line must be placed at the very top of the script. It ensures that the script is executed using Bash, regardless of which shell is currently active.

B. Create the script file with vim editor



C. Making the Script Executable

After creating the file, it must be granted **execute permission** (x) before it can run as a standalone program. use the **chmod** command to modify this attribute.

```
praktikumso@localhost:~/Documents$ chmod +x script.sh
praktikumso@localhost:~/Documents$ ls -l script.sh
-rwxr-xr-x. 1 praktikumso praktikumso 52 Jan 17 14:08 script.sh
```





D. Running the Script

Once executable, the script can be run using a relative or absolute path.

```
praktikumso@localhost:~/Documents$ ./script.sh  
hello, this is my first script!
```

EXERCISE 1 : The Script Foundation (Structure & Variables)

1. Use **vim** to create a file named **hello.sh** and include the correct **shebang** line at the very top.
2. Declare a variable named **GREETING** with the value "Hello World" (remember: no spaces around the **=** sign).
3. Use the **echo** command to print the value of your variable by using the **\$** sign.
4. Save the file, then use the **chmod** command to make it **executable**.
5. Run your script using the **./hello.sh** syntax.

2.2 Variables and Data Handling

Variables in Bash allow you to store and manipulate data efficiently within your automation logic.

A. Variable Declaration

In Bash, variables are created directly without using special keywords.

```
#!/bin/bash  
#  
name="Raven"  
age=19
```

Important Rules:

- **No Spaces:** There must be no spaces before or after the **=** sign.
- **Naming:** Variable names cannot start with a number.
- **Case-Sensitivity:** Variables are case-sensitive (e.g., **Name** is different from **name**).
- **Quotes:** It is best practice to use quotes for string values.

B. Accessing Variable Values

To retrieve or use the value stored in a variable, use the dollar sign (**\$**).



```
#!/bin/bash
#
name="Raven"
age=19

echo $name
echo $age
```

Run the script by run the command:

```
:!bash %
```

```
praktikumso@localhost:~/Documents$ vi contoh.sh

Raven
19

Press ENTER or type command to continue
```

C. System (Environment) Variables

Bash provides built-in variables that store system information and are frequently used in automation:

Built-in variables	Function
\$HOME	Displays the current user's home directory.
\$PATH	Shows the locations where the system looks for commands
\$USER	Displays the name of the active user.

These variables are essential for **system automation**, **multi-user scripting**, and **environment configuration**.

D. Arithmetic Operations

Bash treats variables as strings by default. To perform mathematical calculations, you must use arithmetic expansion with the `$(( ))` syntax.





```
praktikumso@localhost:~/Documents - /usr/bin/vim aritmatika.sh
#!/bin/bash

a=10
b=5
echo $((a+b)) #addition
echo $((a-b)) #subtraction
echo $((a*b)) #multiplication
echo $((a/b)) #division
~
~
~
~
~
-- INSERT -- 8,24 All
```

Note: Without `$(( ))`, Bash will treat the operation as a plain string rather than a calculation.

E. Arrays

Arrays allow you to store more than one value within a single variable name.

```
fruits=("apple" "orange" "mango")

echo ${fruits[0]}
~
```

Output:

```
praktikumso@localhost:~/Documents$ vi arrays.sh

apple

Press ENTER or type command to continue
```

2.3 Input and Output

A. Reading Input and Displaying it

Bash scripts can interact with users by reading input and displaying output, making them dynamic and responsive. The **echo** command is commonly used to display messages or variable values to the terminal, while the **read** command pauses script execution and waits for the user to type information.





```
#!/bin/bash

echo "Enter your name: "
read USERNAME
echo "Hello, $USERNAME"
~
```

When executed, the script will prompt the user to enter input and will produce output as shown below:

```
praktikumso@localhost:~/Documents$ vi inputoutput.sh

Enter your name:
raven
Hello, raven

Press ENTER or type command to continue
```

B. Exit Status and Return Codes

Every script or command in Linux returns an exit status, which is an integer value indicating whether the task was successful. To check exit status, Linux provides a special variable called `$?`, you can check by adding this syntax:

```
echo $?
```

- Success (0): An exit status of 0 means the command or script executed without errors.
- Error (Non-zero): Any value between 1 and 255 indicates an error or a specific failure condition.
- Manual Exit: You can define the exit status of your script using the `exit` command (e.g., `exit 0`).

When you run the script, the exit status will be displayed:

```
praktikumso@localhost:~/Documents$ vi inputoutput.sh

Enter your name:
raven
Hello, raven
0

Press ENTER or type command to continue
```





In automation scripts or module development, the exit status is crucial for making logical decisions. It allows the script to handle errors gracefully; for example, if a command fails (an exit status other than 0), the script can be programmed to stop immediately or send an error notification before proceeding to the next step.

```
#!/bin/bash

mkdir /important_data
if [ $? -ne 0 ]; then
    echo "failed to create directory"
    exit 1
fi

echo "directory created successfully"
```

3. Control Structures in Bash

Control structures are the "brain" of your script. They allow a script to move beyond a simple list of commands by making decisions, repeating processes, and organizing logic into reusable blocks.

3.1 Conditional Statements

Conditional statements enable a script to determine which actions to take based on specific criteria.

A. If Statements

The most basic form of logic; it executes a command only if a condition is **true**.

```
if [condition]; then

fi
```

B. If-else and Elif

Used when there are multiple possibilities or a backup plan if the first condition fails.

```
if [condition]; then
    #if true
elif [condition]; then
    #if second condition is true
else
    #commands if all conditions are false
fi
```

C. Common Comparison Operators

When writing conditions in Bash, you use specific flags inside the square brackets []. Note that spaces are required inside the brackets.





Category	Operator	Description
Numeric	<code>-eq</code>	Equal to
	<code>-ne</code>	Not equal to
	<code>-gt</code>	Greater than
	<code>-lt</code>	Less than
	<code>-ge</code>	Greater than or equal to
	<code>-le</code>	Less than or equal to
File Test	<code>-f</code>	Check if file exists and is a regular file
	<code>-d</code>	Check if directory exists
	<code>-r / -w / -x</code>	Check if file is readable / writable / executable
String	<code>=</code>	Strings are equal
	<code>!=</code>	Strings are not equal
	<code>-z</code>	String is empty (zero length)
	<code>-n</code>	String is not empty

EXERCISE 2: The Logic Master (Conditionals)

1. Write a script named `check_user.sh` that asks the user to input a username.
2. If the input matches your current username, print "Access Granted: Welcome [username]."
3. If the input is `"root"`, print "Warning: Root access detected."
4. Otherwise, print "Access Denied: User not recognized."

3.2 Looping Structures

Loops allow you to run commands repeatedly, saving time and reducing code duplication.

A. For Loop

Best for a known number of iterations, such as processing a list of users or files.

B. While Loop

Runs as long as a condition remains **true**.

C. Until Loop

Runs until a condition becomes **true** (the opposite of while).





EXERCISE 3: The Loop Specialist (Automation & Lists)

Practice using looping structures to perform repetitive tasks automatically.

1. Create a script named `bulk_create.sh`.
2. Use a `for` loop to create 5 directories named `Project_1` through `Project_5`.
3. Inside each directory, automatically create an empty file named `notes.txt`.
4. After running the script, use the `tree` or `ls -R` command to verify the structure.

3.3 Basic Functions

Functions group commands together into a named block, making scripts modular and reusable.

4. Practical Shell Scripting and Automation

Now we are going to focus on applying Bash scripting to real-world scenarios, such as automating system tasks, managing files efficiently, and writing scripts that are secure and easy to maintain.

4.1 Script Arguments

Script arguments allow users to pass input directly when the script is launched, eliminating the need for interactive `read` prompts during execution.

Bash provides a set of special variables known as *positional parameters* that allow a script to receive input directly from the command line when it is executed. These variables make a script more flexible because its behavior can change depending on the arguments supplied by the user.

The variable `$0` represents the name of the script itself, which is useful for displaying usage information or error messages. Arguments passed after the script name are stored sequentially in `$1`, `$2`, `$3`, and so on, where `$1` holds the first argument, `$2` the second, and so forth. When a script needs to access all provided arguments at once, `$@` can be used, while `$#` is available to show how many arguments were supplied in total.

For example, when the command `./backup.sh /home/praktikumso` is executed, the directory path `/home/praktikumso` is stored in `$1`. Inside the script, the command

```
echo "Target Directory : $1"
```

will display the message `Target directory: /home/praktikumso`.

This mechanism is commonly used in practical shell scripting scenarios. Backup scripts often rely on arguments to specify which directory should be backed up. In system administration, arguments allow scripts such as `./create_user.sh nayla` to dynamically create users





based on input. Similarly, monitoring scripts use arguments to define log files, services, or directories that need to be checked.

4.2 Common Automation Tasks

Shell scripting is a primary tool for automating fundamental system administration tasks. Rather than executing commands manually one by one, administrators can group related operations into a single script that runs consistently every time.

Some common automation tasks include:

- **File and directory management**, such as automatically creating directories, copying files, moving data between locations, or removing outdated resources.
- **Permission and ownership management**, where scripts apply `chmod` and `chown` across multiple files or directories to enforce consistent security policies.
- **User and group management**, for example batch-creating user accounts or assigning users to specific groups like `developers`.
- **Log checking**, which involves reading and filtering log files to identify system errors or potential security issues.

The benefits of automation are significant. Scripts reduce the risk of human error during repetitive tasks, save considerable administrative time, and ensure that system configurations remain consistent across different machines and environments.

Shell scripting is more than just a sequence of commands; it is a tool for maintaining system consistency. Below are real-world scenarios and script examples commonly used by administrators:

A. Automated User Onboarding

Administrators often need to create multiple accounts. This script speeds up the process by creating a user, setting `#!/bin/bash`

```
USERNAME=$1
GROUP=$2
# Create group if it doesn't exist [cite: 701]
groupadd $GROUP 2>/dev/null

# Create user with a home directory and assign to group [cite: 692]
useradd -m -g $GROUP $USERNAME
echo "User $USERNAME has been added to group $GROUP." a home directory, and assigning them to
a group automatically.
```



B. Log Monitoring and Alerting

Scripts are used to filter log files to quickly identify errors or suspicious activity.

```
#!/bin/bash
# Script to check for "failed" attempts in auth log
LOG_FILE="/var/log/auth.log"
SEARCH_TERM="failed"
echo "Searching for $SEARCH_TERM in $LOG_FILE..."
grep -i "$SEARCH_TERM" $LOG_FILE > failed_attempts.txt
echo "Summary saved to failed_attempts.txt"
```

C. Batch Permission Enforcement

This ensures all files within a directory follow a uniform security policy.

```
#!/bin/bash
# Enforce permissions: 755 for directories, 644 for files
TARGET_DIR=$1

find $TARGET_DIR -type d -exec chmod 755 {} +
find $TARGET_DIR -type f -exec chmod 644 {} +
echo "Permissions for $TARGET_DIR have been standardized."
```

EXERCISE 4: The System Monitor (Arguments & Functions)

1. Write a script named `sys_report.sh` that accepts one argument: `disk`, `user`, or `all`.
2. If the argument is `disk`, show the current storage usage.
3. If the argument is `user`, show the current logged-in user and their groups.
4. If the argument is `all`, perform both tasks.
5. Wrap each reporting task (disk check and user check) inside a basic function.

4.3 Debugging and Best Practices

In shell scripting, errors may occur due to incorrect syntax, unexpected input, or failed commands. Debugging helps identify where and why a script fails, while best practices ensure that scripts are reliable, secure, and easy to maintain over time.

One of the simplest ways to debug a Bash script is by enabling execution tracing. By using the `set -x` option, Bash will display each command before it is executed, along with its expanded variables. This allows the script writer to follow the execution flow and detect incorrect logic or unexpected values.

Example:

```
#!/bin/bash
set -x
NAME="Rocky"
echo "Hello $NAME"
```



Output:

```
praktikumso@localhost:~$ ./coba.sh
+ NAME=Rocky
+ echo 'Hello Rocky'
Hello Rocky
```

Another useful option is `set -e`, which forces the script to stop immediately if any command returns a non-zero exit status. This behavior is important in automation tasks where continuing after a failure could lead to data corruption or inconsistent system states.

```
#!/bin/bash
set -e
# Perintah yang salah (misal: direktori tidak ada)
cd /folder_yang_tidak_ada
# Perintah di bawah ini tidak akan pernah dijalankan karena set -e
echo "Proses selesai"
```

In addition, checking exit status using `$?` can help determine whether a command executed successfully. This is often used to make decisions or display meaningful error messages.

4.4 Auto-run Scripts (System Automation)

For automation to be truly effective, scripts often need to run automatically without manual intervention. In Linux, there are several ways to achieve this:

A. Scheduling with Cron (Periodic Tasks)

Cron is a time-based job scheduling daemon in Linux that allows you to execute scripts automatically at specific intervals. To manage these schedules, users interact with a configuration file called the **crontab** (cron table) using the `crontab -e` command.

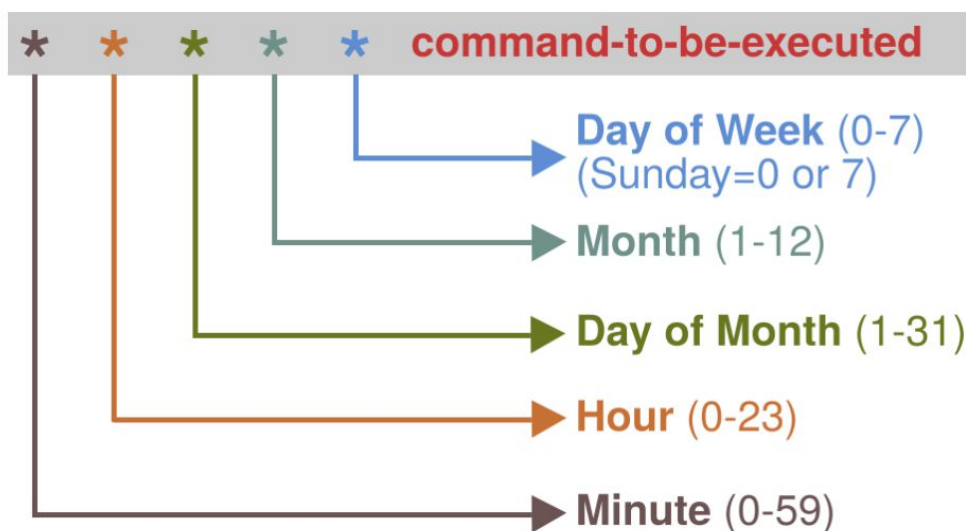


Fig 1. Crontab Schedule Format





Example: Daily Backup at Midnight

```
0 0 * * * /home/user/scripts/backup.sh
```

It will run at Minute 0, Hour 0, every day of the month, every month, and every day of the week.

B. Startup Scripts with .bashrc (Login Tasks)

If you want a script to run every time a user logs in or opens a new terminal:

1. Open the `.bashrc` file in the home directory: `vi ~/.bashrc`.
2. Add the script path at the very bottom: `/path/to/your_script.sh`.
3. Apply changes by running: `source ~/.bashrc`.

C. Systemd Services (Background Tasks)

For scripts that must run continuously in the background starting from system boot:

1. Create a service unit file in `/etc/systemd/system/myscript.service`.
2. Use the command `systemctl enable myscript.service` to start it automatically upon booting.

EXERCISE 5: The Automator (Cron & Automation)

1. Add the `set -x` command at the beginning of any script to enable **execution tracing** (debugging mode).
2. Open the `crontab` editor using the `crontab -e` command.
3. Add a line to schedule a script to run **every minute** (using the `* * * * *` format).
4. Verify the automation by checking if the script's output (like a created file or log entry) appears as expected.





PRACTICAL APPLICATIONS

PRACTICE

Objective: Students directly try basic commands for simple file navigation and manipulation.
(The exercise is not shown yet; students wait for instructions during the practice session.)

DEMO - CASE STUDY

The Situation

You are a Senior System Administrator at an enterprise data center. The team manages hundreds of Rocky Linux servers where environment setups (creating project workspaces, logs, and audit trails) are currently done manually. This manual process is no longer scalable for the upcoming deployment of a large-scale microservices architecture.

The Problem to Identify

The lack of automation causes inconsistent directory structures across different servers. Existing attempts at scripting fail because they do not handle multiple inputs, ignore system permissions (UID/GID), and do not provide an audit trail of what was created. Furthermore, if a script fails mid-execution (e.g., due to a full disk), it provides no exit status, making it impossible for centralized orchestration tools to detect the failure.

The Mission

Your mission is to develop a robust **Automation Script** that:

1. **Multi-Argument Handling:** Accepts an unlimited number of directory names as command-line arguments using loops.
 2. **Input & Permission Validation:** Checks if each input is empty and verifies if the current user has the authority to create directories in the target path.
 3. **Automated Workspace Setup:** For every valid directory created, the script must automatically generate an `audit.log` inside it, containing the creation timestamp and the user ID who initiated the process.
 4. **Exit Status & Debugging:** Implementation of proper Exit Codes (0 for success, non-zero for failure) so the system can track automation results.
 5. **Direct Execution:** The script must be made executable and run directly via its path (`./`) to demonstrate proper environment handling.
-

Demo Rules

1. **No Module or Notes Access** Students are strictly prohibited from opening the module PDF, personal notes, or any online documentation during the demo session.
2. **Vim-Only Scripting** The entire process of writing and editing scripts must be performed using the **Vim** text editor. The use of GUI editors or other text editors is not permitted.





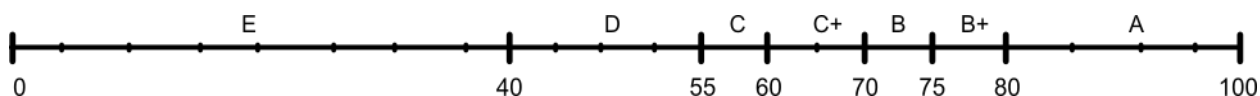
3. **Demonstrate Core Bash Skills** Students must clearly show proficiency in:
 - Script Structure
 - Execution Mastery
 - Variables & Input
 - Logic & Automation
 - I/O & Error Handling
4. **Final Verification** After executing the script, students must verify the automation results
5. **Live Conceptual Assessment** The assistant will conduct a Q&A session to evaluate your **Conceptual Understanding (40%)**. Be prepared to explain the logic behind every line of code you write, as this significantly impacts your final grade.
6. **Follow Live Instructions** All actions must be performed interactively following the directions of the instructor or laboratory assistant.



ASSESSMENT RUBRIC

Assessment Aspect	Points (Total 100%)
EXERCISE	Total 5%
Exercise 1	1%
Exercise 2	1%
Exercise 3	1%
Exercise 4	1%
Exercise 5	1%
PRACTICE	Total 20%
Practice 1	10%
Practice 2	10%
DEMO	Total 75%
Case Study Completion	35%
Basic Bash Scripting	15%
Variable & Logic Mastery	15%
Execution & Permission Mastery	10%

ASSESSMENT SCALE



- A** = (81 - 100) → Sepuh
- B+** = (75 - 80) → Very Good
- B** = (70 - 74) → Good
- C+** = (60 - 69) → Fairly Good
- C** = (55 - 59) → Fair
- D** = (41 - 54) → Poor
- E** = (0 - 40) → Bro really...

